



TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO KẾT THÚC MÔN CÔNG NGHỆ PHẦN MỀM ĐỀ TÀI:

“

BeeLife Ventures
Nền tảng thương mại điện tử cho ngành
Nuôi ong Thông minh

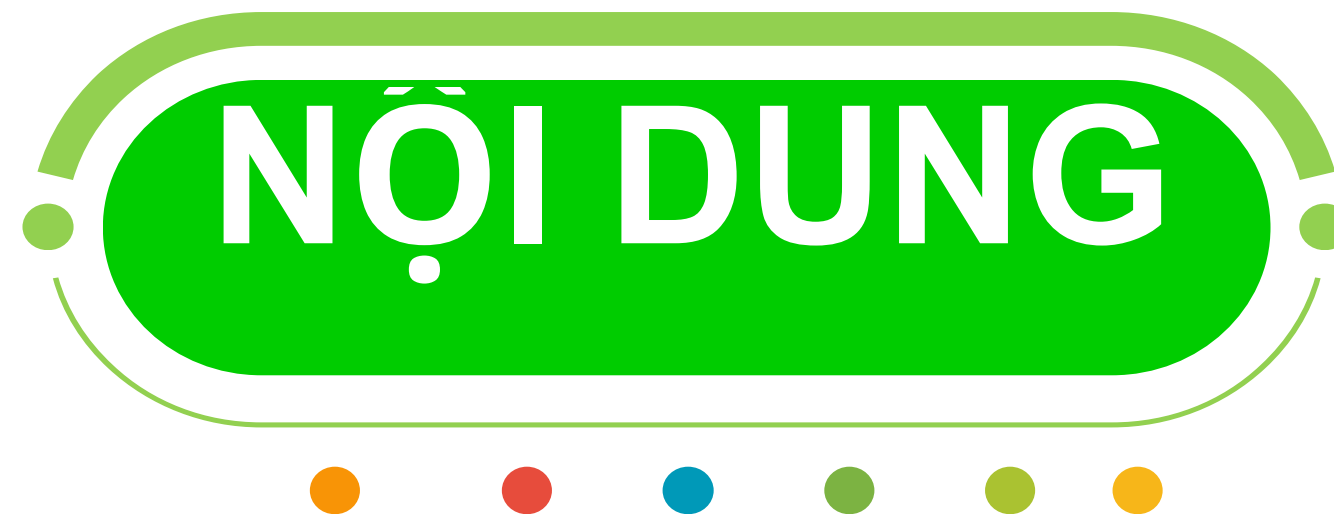
”

GVHD: TS Nguyễn Bảo Ân
SVTH:

Trần Minh Điền
Nguyễn Văn Hoàng
Nguyễn Lê Duy



NỘI DUNG



- Phần 1** Giới thiệu và khái niệm cơ bản về công nghệ phần mềm
- Phần 2** Mô hình phát triển phần mềm
- Phần 3** Mô hình phát triển phần mềm Agile và kế hoạch phát triển
- Phần 4** Kiến trúc phần mềm dựa trên đám mây
- Phần 5** Design Patterns và Architectural Patterns
- Phần 6** Công cụ và ngôn ngữ lập trình
- Phần 7** Bảo mật và an toàn phần mềm
- Phần 8** Demo sản phẩm



Phần 1: Tổng quan dự án

1.1

Tổng quan dự án

1.2

Đặc tả nhu cầu ứng dụng

1.3

Tính cần thiết của phần mềm
phân tán/microservices/cloud





- **Tên dự án: BeeLife Ventures**
- **Mô tả: Nền tảng thương mại điện tử cho ngành nuôi ong thông minh**
- **Mục đích: Kết nối người nuôi ong với người tiêu dùng, cung cấp sản phẩm mật ong chất lượng cao và hỗ trợ cộng đồng người nuôi ong**

- Đối tượng người dùng:
 - Người tiêu dùng: Mua sản phẩm từ mật ong
 - Người nuôi ong: Cung cấp sản phẩm và thông tin
- Quản trị viên: Quản lý hệ thống
- Thách thức thực tế:
 - Ngành nuôi ong Việt Nam đang phát triển nhưng thiếu nền tảng công nghệ hỗ trợ
 - Nhu cầu kết nối trực tiếp giữa người sản xuất và người tiêu dùng

Khả năng mở rộng

Đáp ứng nhu cầu tăng trưởng người dùng và sản phẩm

Phân tán địa lý

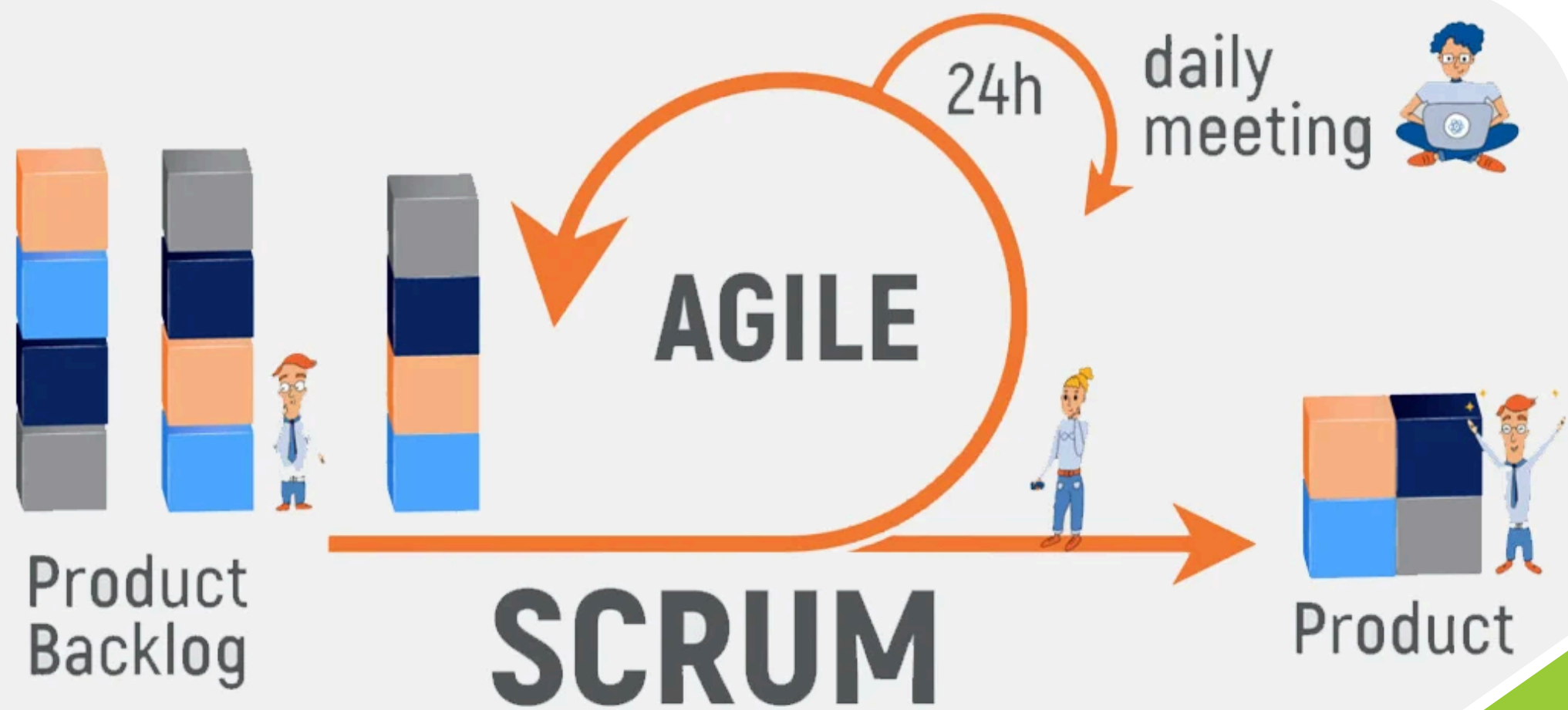
Đáp ứng nhu cầu tăng trưởng người dùng và sản phẩm

Tính sẵn sàng cao

Giảm thiểu thời gian chết của hệ thống

Độc lập triển khai

Đáp ứng nhu cầu tăng trưởng người dùng và sản phẩm



Phần 2: Mô hình phát triển phần mềm

2.1

Agile và Scrum

2.2

Phát triển phần mềm dựa trên đám mây

- **Lý do chọn Agile:**

- **Thích ứng với thay đổi yêu cầu**
- **Tạo giá trị sớm và liên tục**
- **Phù hợp với nhóm nhỏ có thời gian hạn chế (2 tuần)**

- **Áp dụng Scrum:**

- **Sprint 2 tuần**
-

2.2 Phát triển phần mềm dựa trên đám mây

- Containerization với Docker
- CI/CD tự động
- Triển khai trên VPS
- RESTful API cho giao tiếp giữa frontend và backend





Phần 3: Mô hình phát triển phần mềm Agile

3.1

Product Backlog

3.2

Phân chia Sprint và Sprint Backlog

3.3

Phân công thành viên và kế hoạch

3.3

Burndown Chart

- Các epic chính:
 - Epic 1: Phân tích & Thiết kế kiến trúc phần mềm (14 điểm)
 - Epic 2: Phát triển Backend (13 điểm)
 - Epic 3: Phát triển Frontend (12 điểm)
 - Epic 4: DevOps & CI/CD (11 điểm)
 - Epic 5: Kiểm thử & Hoàn thiện (6 điểm)
 - Epic 6: Quản lý dự án & Scrum (4 điểm)



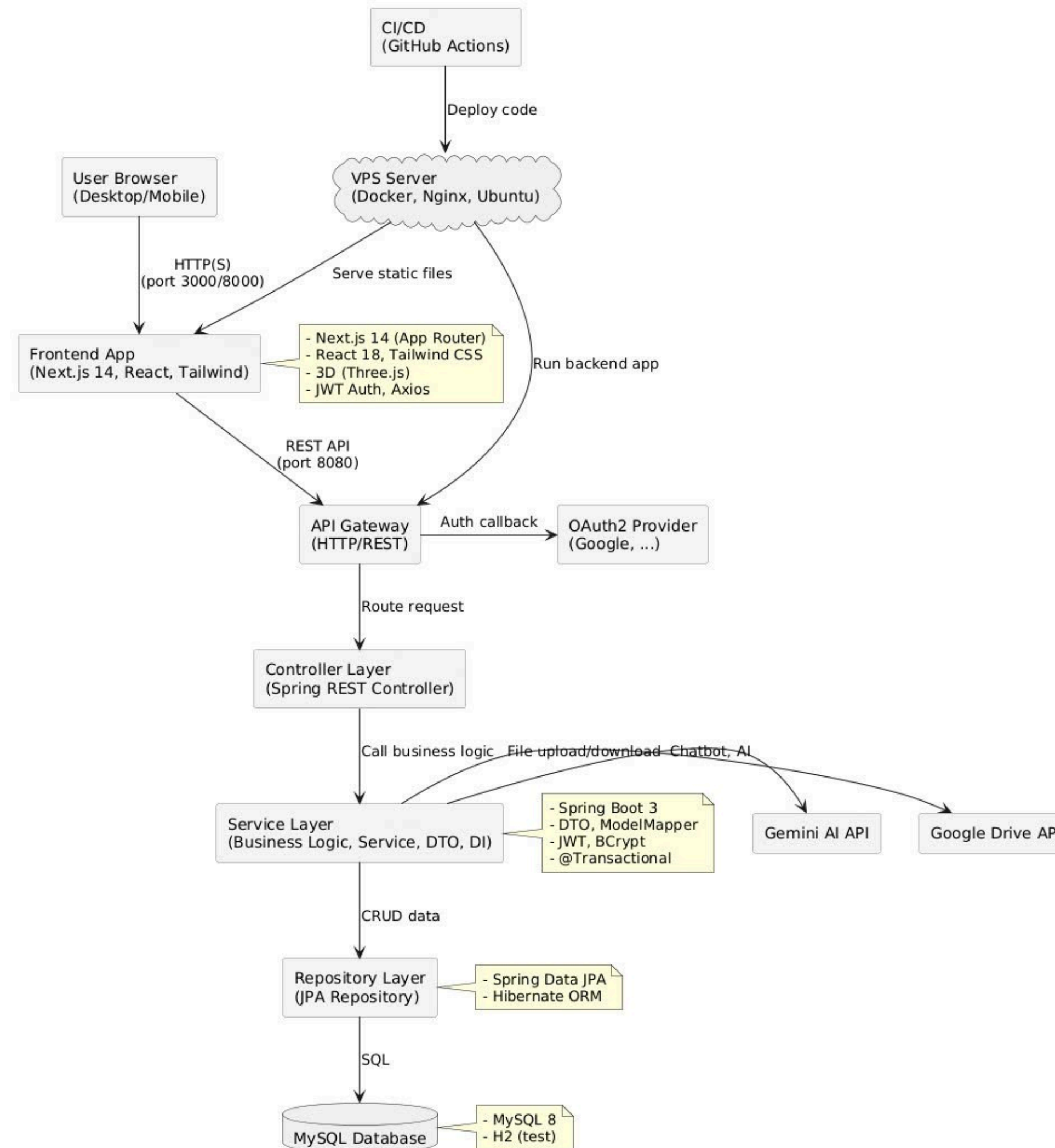
Phân chia Sprint và Sprint Backlog

- Sprint 1 (18 điểm): Phân tích, thiết kế, thiết lập nền tảng
- Sprint 2 (19 điểm): Phát triển chức năng cốt lõi và thiết lập Docker
- Sprint 3 (15 điểm): Tính năng nâng cao, unit test, CI/CD
- Sprint 4 (6 điểm): Kiểm thử tích hợp, sửa lỗi, tài liệu, demo

- Trần Minh Điền: DevOps & Lập trình viên Backend
- Nguyễn Văn Hoàng: Trưởng nhóm Backend
- Nguyễn Lê Duy: Trưởng nhóm Frontend



Phần 4: Kiến trúc phần mềm dựa trên đám mây



- **API Endpoints chính:**
 - **/api/v1/auth:** Đăng ký, đăng nhập
 - **/api/v1/products:** CRUD sản phẩm
 - **/api/v1/cart:** Quản lý giỏ hàng
 - **/api/v1/orders:** Quản lý đơn hàng
 - **/api/v1/admin:** Quản lý hệ thống
- **Tài liệu API:** Swagger/OpenAPI 3





Phần 5: Design Patterns và Architectural Patterns

- **Dependency Injection (IoC):**

- Sử dụng Spring Framework's IoC container
- Giảm sự phụ thuộc giữa các components
- Dễ dàng thay thế, mở rộng và kiểm thử

- **Builder Pattern:**

- Sử dụng trong các DTO (Data Transfer Objects)
- Tạo đối tượng phức tạp qua từng bước

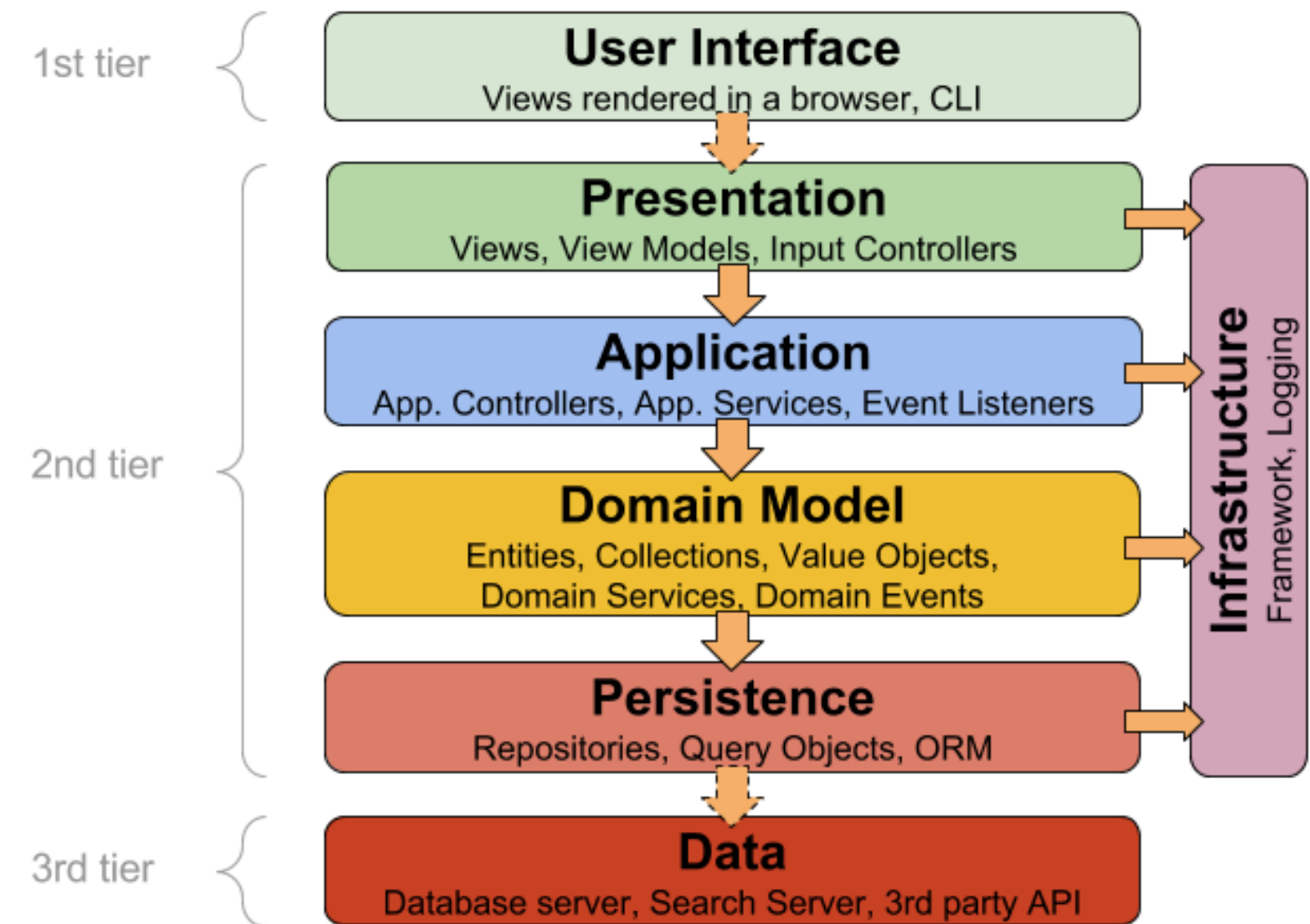
- **Repository Pattern:**

- Sử dụng Spring Data JPA
- Trừu tượng hóa việc truy cập dữ liệu
- Tách biệt business logic và data access logic

- **Factory Pattern:**

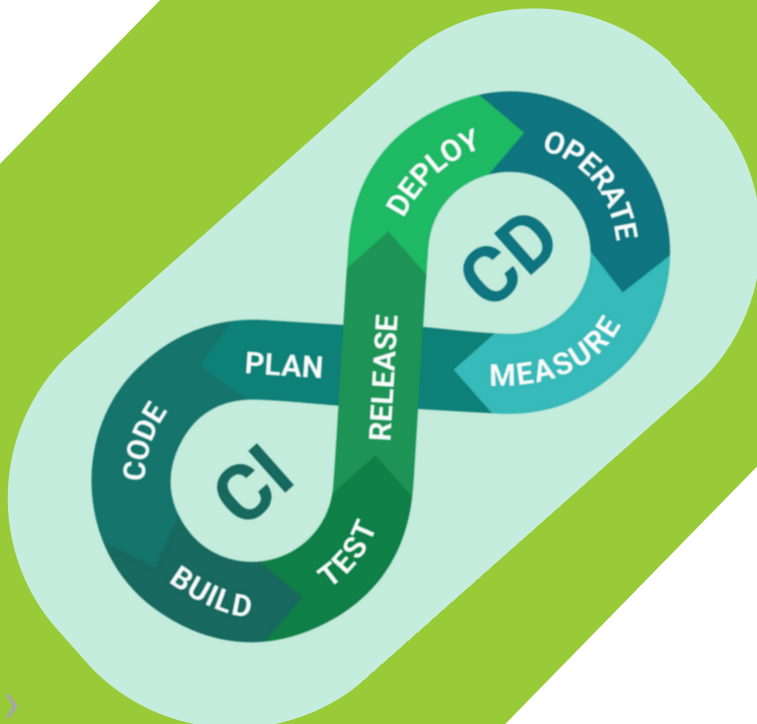
- ModelMapperConfig tạo và cấu hình ModelMapper
- BeanFactory trong Spring container

- **Presentation Layer:** Controllers
- **Business Layer:** Services
- **Data Access Layer:** Repositories
- **Domain Layer:** Entities



www.herbertograca.com

Phần 6: Công cụ và ngôn ngữ lập trình



6.1

Ngôn ngữ lập trình và frameworks

6.2

Sử dụng Git/GitHub

6.3

Sử dụng Docker

6.4

Triển khai lên host

- Backend:

- Java 17: Ngôn ngữ lập trình chính cho backend
- Spring Boot 3: Framework phát triển ứng dụng
- Spring Cloud: Hỗ trợ phát triển microservices
- Spring Security: Xác thực và phân quyền
- Spring Data JPA: Truy xuất dữ liệu
- Hibernate ORM: Ánh xạ đối tượng quan hệ
- ModelMapper: Chuyển đổi giữa Entity và DTO

- Frontend:

- TypeScript: Ngôn ngữ lập trình chính cho frontend
- Next.js 14: Framework React với SSR và SSG
- React: Thư viện UI
- Tailwind CSS: Framework CSS
- React Hook Form: Quản lý form
- Zustand: Quản lý state

- Database:

- MySQL 8.0: Cơ sở dữ liệu quan hệ
- SQL: Ngôn ngữ truy vấn dữ liệu





- Quy trình phát triển:
 - 1.Feature branches
 - 2.Pull Request và Code Review
 - 3.Chỉ merge vào main khi release



```
# Giai đoạn 1: Xây dựng ứng dụng
FROM maven:3.8-eclipse-temurin-17 AS builder
WORKDIR /build
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

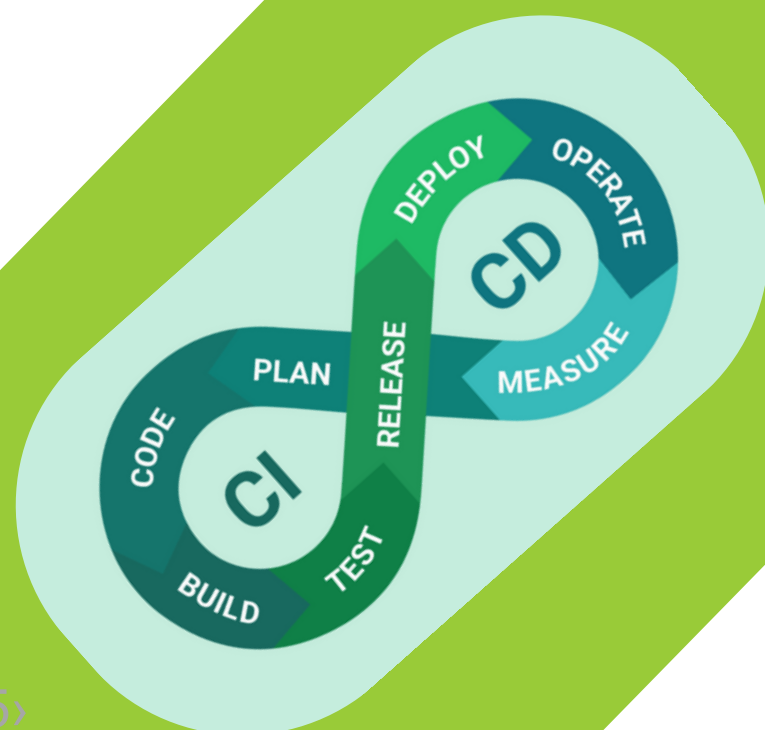
# Giai đoạn 2: Tạo image runtime
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN apk add --no-cache curl
COPY --from=builder /build/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```


- Quy trình CI/CD tự động với GitHub Actions:
 - Kiểm thử và build Frontend
 - Kiểm thử và build Backend
 - Kiểm tra Docker build
 - Deploy lên VPS khi merge vào main
- Auto-deploy flow:
 - Backup phiên bản hiện tại
 - Pull code mới nhất
 - Rebuild và restart containers
 - Health check để đảm bảo hệ thống hoạt động
 - Rollback tự động nếu có lỗi





Phần 7: Bảo mật và an toàn phần mềm



- **Xác thực và phân quyền**

- JWT Authentication: JSON Web Token cho xác thực stateless
- Spring Security: Bảo vệ API endpoints dựa trên vai trò
- Password Hashing: BCrypt với salt

- **Bảo vệ dữ liệu**

- HTTPS: Mã hóa dữ liệu trên đường truyền
- Input Validation: Kiểm tra đầu vào ở cả client và server
- SQL Injection Protection: JPA/Hibernate

- **Kiểm thử bảo mật**

- Unit Testing: JUnit cho backend, Jest cho frontend
- Integration Testing: Kiểm thử kết hợp các thành phần



SQL Injection



DEMO

<https://zettix.net/>